

Comparison of Sensor-Conditioned and Dynamics-Learning Physics-Informed Neural Networks for Fixed-Wing UAV State Estimation

Nafis Mohammad Nayeem, Student Member, IEEE, and Dr. Martin Hagan, Member, IEEE

Abstract—Accurate state estimation for small fixed-wing unmanned aerial vehicles (UAVs) is difficult when the estimator must generalize from maneuver-rich synthetic data to a held-out flight profile while using noisy inertial and navigation-like measurements. This paper compares two physics-informed neural-network (PINN) formulations implemented on the same 6-degree-of-freedom fixed-wing simulation environment. The first is a sensor-conditioned PINN observer that maps normalized time, measured accelerations and angular rates, noisy measurement channels, control-surface commands, and propeller speed directly to a continuous 12-state estimate. The second is a dynamics-learning PINN that learns a residual correction to a reduced observer-physics transition and is evaluated in one-step and free-rollout modes against extended Kalman filter (EKF) and invariant/iterated EKF (IEKF) baselines. Both models use residual multilayer perceptrons with width 256 and six residual blocks, robust Huber-style losses, curriculum training, Adam optimization [12], and L-BFGS-B fine tuning [13]. The comparison shows that the dynamics-learning free rollout gives lower inertial-position and Euler-attitude RMSE on the held-out turn, whereas the sensor-conditioned observer gives substantially lower wind-component RMSE and lower lateral/vertical body-velocity RMSE. The results indicate that direct sensor conditioning and transition learning solve complementary parts of the estimation problem: measurement-conditioned reconstruction is useful for algebraic sensor consistency and wind recovery, while physics-corrected rollout better preserves attitude and position evolution over the held-out maneuver. Limitations include synthetic-data dependence, no learned uncertainty, Euler-angle singularities, computational cost, and sensitivity to rollout design.

Index Terms—Physics-informed neural networks, UAV state estimation, neural observer, fixed-wing aircraft, sensor fusion, dynamics learning, EKF, IEKF, residual networks, curriculum learning.

I. INTRODUCTION

SMALL fixed-wing UAVs rely on lightweight sensors and estimators to reconstruct position, attitude, translational velocity, and wind-related states. Classical filters, especially EKF-family methods, remain attractive because they recursively propagate a physical model and update it with measurement residuals. Their performance, however, depends on the fidelity of the assumed dynamics, the quality of linearization, and the selected process and measurement covariances. Purely data-driven neural estimators can approximate nonlinear state maps but may violate kinematic identities or drift when asked

to extrapolate to a maneuver that differs from the training profiles.

Physics-informed neural networks (PINNs) provide a mechanism to combine data and differential-equation structure. In state estimation, the key question is not merely whether a neural network can fit a trajectory, but how the network should be conditioned and how the physics should enter the loss. The two implementations compared here represent two distinct answers. The sensor-conditioned observer directly predicts the state from time, sensor channels, and control channels. The dynamics-learning model learns a correction to the known reduced observer transition and is evaluated as a predictive dynamics model.

The contributions of this paper are fourfold. First, it formulates a common fixed-wing UAV estimation problem for both codes, including the 15-state simulator, 12-state reduced observer state, 10-channel measurement map, and measured acceleration/rate input. Second, it derives and contrasts the two neural architectures: a direct sensor-conditioned continuous-time observer and a hybrid residual transition model. Third, it analyzes the loss components, training choices, and parameter values used by both implementations. Fourth, it compares the held-out turn results using the supplied plots, identifies where each formulation performs well, and discusses limitations and directions for a more flight-ready neural observer.

II. LITERATURE REVIEW

A. Classical Nonlinear Estimation for Flight Systems

Kalman filtering and nonlinear variants are standard tools for aerospace navigation and state estimation [3], [4]. The EKF uses a first-order local linearization of the nonlinear process and measurement functions, while iterated and invariant variants can improve update consistency in nonlinear or geometric settings. These methods are interpretable and computationally recursive, but performance is tied to process-model assumptions, sensor-noise tuning, and local observability.

B. Physics-Informed Neural Networks and Neural Dynamics

PINNs penalize differential-equation residuals while fitting data, enabling neural approximations that respect known physical constraints [1]. Neural ordinary differential equations and residual dynamics models similarly combine continuous-time or discrete-time dynamics with trainable function approximators [5]. For UAV estimation, a central design decision is whether the neural network should output the state directly,

output the dynamics, or output a correction to a known dynamics model.

C. Architectural Components for Maneuver-Rich Data

Fourier time features help MLPs represent oscillatory responses caused by chirps, circles, and multi-sine controls [6]. Residual blocks improve trainability by letting deeper networks learn corrections to hidden features rather than entirely new transformations at each layer [8]. Layer normalization stabilizes the residual blocks when the loss combines supervised, physics, measurement, and regularization terms with different magnitudes. The two codes use these ideas differently: the sensor-conditioned model encodes time with Fourier features, whereas the dynamics-learning model omits time and treats the transition as Markovian in state and input.

III. PROBLEM SETUP AND EQUATIONS

A. Simulator and Observer States

The full fixed-wing simulator state is

$$x_{15} = [p_N, p_E, p_D, \phi, \theta, \psi, u, v, w, p, q, r, V_{wx}, V_{wy}, V_{wz}]^T, \quad (1)$$

where (p_N, p_E, p_D) is inertial position, (ϕ, θ, ψ) are Euler angles, (u, v, w) are body-axis velocities, (p, q, r) are body angular rates, and (V_{wx}, V_{wy}, V_{wz}) are inertial wind components. Both neural formulations estimate the 12-state observer vector

$$x = x_{12} = [p_N, p_E, p_D, \phi, \theta, \psi, u, v, w, V_{wx}, V_{wy}, V_{wz}]^T. \quad (2)$$

The angular rates are not estimated states; they enter through the measured observer input

$$u_m = [A_{mx}, A_{my}, A_{mz}, p_m, q_m, r_m]^T \in \mathbb{R}^6. \quad (3)$$

The auxiliary control vector is

$$\eta = [\delta_a, \delta_e, \delta_r, n_p]^T, \quad (4)$$

where δ_a , δ_e , and δ_r are control-surface deflections and n_p is propeller rotational speed.

B. Kinematics

The body-to-inertial rotation matrix used in both scripts is

$$R_{IB} = \begin{bmatrix} c_\psi c_\theta & c_\psi s_\theta s_\phi - s_\psi c_\phi & c_\psi s_\theta c_\phi + s_\psi s_\phi \\ s_\psi c_\theta & s_\psi s_\theta s_\phi + c_\psi c_\phi & s_\psi s_\theta c_\phi - c_\psi s_\phi \\ -s_\theta & c_\theta s_\phi & c_\theta c_\phi \end{bmatrix}, \quad (5)$$

where $c_a = \cos a$ and $s_a = \sin a$. Position evolves according to

$$\dot{p} = R_{IB} [u \ v \ w]^T + [V_{wx} \ V_{wy} \ V_{wz}]^T. \quad (6)$$

The scalar components are

$$\dot{p}_N = V_{wx} - v(c_\theta s_\psi - c_\psi s_\theta s_\phi) + w(s_\theta s_\psi + c_\phi c_\psi s_\theta) + u c_\theta c_\psi, \quad (7)$$

$$\dot{p}_E = V_{wy} + v(c_\phi c_\psi + s_\phi s_\theta s_\psi) - w(c_\psi s_\phi - c_\phi s_\theta s_\psi) + u c_\theta s_\psi, \quad (8)$$

$$\dot{p}_D = V_{wz} - u s_\theta + w c_\phi c_\theta + v c_\theta s_\phi. \quad (9)$$

The Euler-rate equations are

$$\dot{\phi} = p + q \sin \phi \tan \theta + r \cos \phi \tan \theta, \quad (10)$$

$$\dot{\theta} = q \cos \phi - r \sin \phi, \quad (11)$$

$$\dot{\psi} = \frac{q \sin \phi + r \cos \phi}{\cos \theta}, \quad (12)$$

with implementation-level clamping of $\cos \theta$ to avoid division by zero.

C. Reduced Observer Dynamics

The reduced observer dynamics use measured accelerations and measured rates instead of reconstructing all aerodynamic forces:

$$\dot{u} = A_{mx} - qw + rv - g \sin \theta, \quad (13)$$

$$\dot{v} = A_{my} + pw - ru + g \cos \theta \sin \phi, \quad (14)$$

$$\dot{w} = A_{mz} - pv + qu + g \cos \phi \cos \theta, \quad (15)$$

$$\dot{V}_{wx} = 0, \quad \dot{V}_{wy} = 0, \quad \dot{V}_{wz} = 0. \quad (16)$$

Equations (6)-(16) define

$$\dot{x} = f_o(x, u_m), \quad (17)$$

which is the common reduced-physics model used by both PINN formulations.

D. Measurement Map

The measurement vector is

$$z = [p_N, p_E, p_D, \phi, \theta, \psi, V_N^I, V_E^I, V_D^I, V_t]^T \in \mathbb{R}^{10}. \quad (18)$$

The inertial velocity channels and airspeed are

$$[V_N^I, V_E^I, V_D^I]^T = R_{IB}[u, v, w]^T + [V_{wx}, V_{wy}, V_{wz}]^T, \quad (19)$$

$$V_t = \sqrt{u^2 + v^2 + w^2 + \epsilon}, \quad \epsilon = 10^{-8}. \quad (20)$$

Therefore

$$z = h(x), \quad z_m = h(x) + \varepsilon_z, \quad (21)$$

with Euler-angle residuals wrapped to $(-\pi, \pi]$.

E. Aerodynamic Simulator for Synthetic Truth

The truth trajectories are generated by a 15-state fixed-wing simulator with aerodynamic and propulsive forces following standard fixed-wing modeling practice [9]–[11]. Define

$$V_t = \sqrt{u^2 + v^2 + w^2}, \quad \alpha = \text{atan2}(w, u), \quad \beta = \sin^{-1}(v/V_t), \quad (22)$$

$$\hat{p} = \frac{pb}{2V_t}, \quad \hat{q} = \frac{qc}{2V_t}, \quad \hat{r} = \frac{rb}{2V_t}, \quad J = \frac{V_t}{n_p D_p}. \quad (23)$$

The coefficient model is

$$C_J = C_{J0} + C_{JJ}J + C_{J2}J^2, \quad (24)$$

$$C_X = C_{X0} + C_{Xq}\hat{q} + C_{X\delta_e}\delta_e + C_{X\alpha}\alpha + C_{X\alpha^2}\alpha^2, \quad (25)$$

$$C_Y = C_{Y0} + C_{Yp}\hat{p} + C_{Yr}\hat{r} + C_{Y\delta_a}\delta_a + C_{Y\delta_r}\delta_r + C_{Y\beta}\beta, \quad (26)$$

$$C_Z = C_{Z0} + C_{Zq}\hat{q} + C_{Z\delta_e}\delta_e + C_{Z\alpha}\alpha, \quad (27)$$

$$C_l = C_{l0} + C_{lp}\hat{p} + C_{lr}\hat{r} + C_{l\delta_a}\delta_a + C_{l\delta_r}\delta_r + C_{l\beta}\beta, \quad (28)$$

$$C_m = C_{m0} + C_{mq}\hat{q} + C_{m\delta_e}\delta_e + C_{m\alpha}\alpha, \quad (29)$$

$$C_n = C_{n0} + C_{np}\hat{p} + C_{nr}\hat{r} + C_{n\delta_a}\delta_a + C_{n\delta_r}\delta_r + C_{n\beta}\beta. \quad (30)$$

With dynamic pressure $\bar{q} = \frac{1}{2}\rho V_t^2$, body forces and moments are

$$F_X = \bar{q}SC_X + D_p^4 \rho \eta_p n_{prop} n_p^2 C_J, \quad (31)$$

$$F_Y = \bar{q}SC_Y, \quad F_Z = \bar{q}SC_Z, \quad (32)$$

$$M = [\bar{q}SbC_l, \bar{q}ScC_m, \bar{q}SbC_n]^T. \quad (33)$$

The simulator integrates

$$\dot{p} = R_{IB}v_b + V_w, \quad (34)$$

$$\dot{\Theta} = L(\phi, \theta)\omega, \quad (35)$$

$$\dot{v}_b = \frac{1}{m}(F_{aero/prop} + R_{IB}^T[0, 0, mg]^T) + v_b \times \omega, \quad (36)$$

$$\dot{\omega} = I^{-1}(M + (I\omega) \times \omega), \quad (37)$$

$$\dot{V}_w = 0. \quad (38)$$

A fixed-step fourth-order Runge–Kutta solver with $T_s = 0.05$ s generates the synthetic data.

IV. DATA TYPES, DATASET CONSTRUCTION, AND PARAMETER CHOICES

A. Data Types and Shapes

Both codes use NumPy arrays for simulation and PyTorch `torch.float32` tensors for learning. Each maneuver has $N = T_f/T_s + 1 = 2401$ samples with $T_f = 120$ s and $T_s = 0.05$ s. The validation split is a contiguous 15% tail, giving 2041 training samples and 360 validation samples per maneuver before batching.

The sensor-conditioned model uses the base input

$$X_{SC} = [\tau, u_m, z_m, \delta_a, \delta_e, \delta_r, n_p] \in \mathbb{R}^{21}. \quad (39)$$

Fourier encoding expands the single time coordinate with $K = 4$ frequencies, so the encoded dimension is $1 + 2K + 20 = 29$. The dynamics-learning model uses a Markovian transition input

$$X_D = [\tilde{x}_k, \tilde{u}_{m,k}] \in \mathbb{R}^{18}, \quad (40)$$

where tildes denote standardized variables.

B. Rationale for Parameter Choices

The 20 Hz sample rate resolves the scripted control inputs while keeping automatic differentiation and EKF/IEKF comparisons computationally tractable. The 120 s horizon gives enough time for heading evolution, circular motion, and rich multi-sine excitation. The use of contiguous validation tails avoids random leakage across adjacent time samples and evaluates end-of-trajectory generalization within each maneuver. Width 256 and six residual blocks provide sufficient capacity for the nonlinear state map and transition correction while remaining trainable under derivative constraints. Robust residual penalties reduce the sensitivity of state, measurement, and physics terms to high residuals caused by finite differencing, derivative amplification, or transient mismatch.

V. ARCHITECTURE DETAILS

A. Sensor-Conditioned PINN Observer

The direct observer implements

$$\hat{x}_{12}(t) = f_\theta(\tau(t), u_m(t), z_m(t), \eta(t)). \quad (41)$$

The Fourier time encoding is

$$\gamma(\tau) = [\tau, \sin(\pi\tau), \cos(\pi\tau), \dots, \sin(2^{K-1}\pi\tau), \cos(2^{K-1}\pi\tau)]^T. \quad (42)$$

For residual block ℓ ,

$$h_{\ell+1} = \text{LN}(h_\ell + W_{\ell,2}\sigma(W_{\ell,1}h_\ell + b_{\ell,1}) + b_{\ell,2}), \quad (43)$$

where σ is selected from tanh, SiLU, Swish, and sine by validation loss. The network has one trainable state-output head; the predicted measurement is not a second trainable head but an analytic map $\hat{z} = h(\hat{x})$.

Automatic differentiation produces

$$\dot{\hat{x}} = \frac{1}{T_f} \frac{\partial \hat{x}}{\partial \tau}, \quad (44)$$

which is used in the continuous-time residual $\dot{\hat{x}} - f_o(\hat{x}, u_m)$.

B. Dynamics-Learning Hybrid Transition PINN

The dynamics-learning model is intentionally not a direct observer. It learns a residual correction to an explicit reduced-physics transition:

$$x_{phys,k+1} = x_k + T_s f_o(x_k, u_{m,k}), \quad (45)$$

$$\Delta x_{NN,k} = g_\theta(\tilde{x}_k, \tilde{u}_{m,k}), \quad (46)$$

$$\hat{x}_{k+1} = x_{phys,k+1} + \Delta x_{NN,k}. \quad (47)$$

The output of g_θ is normalized as a correction and then denormalized using correction statistics computed from the training data. Because the model is a transition law, it can be evaluated in one-step mode using truth states as inputs or in free-rollout mode using its own predicted states.

C. Architecture Comparison

Fig. 1 summarizes the two architectures. The important distinction is the information pattern: the sensor-conditioned observer is a measurement-conditioned reconstruction map, while the dynamics-learning PINN is a model-correction map designed for sequence propagation.

VI. LOSS FUNCTIONS AND ERROR COMPONENTS

A. Robust Residual Penalty

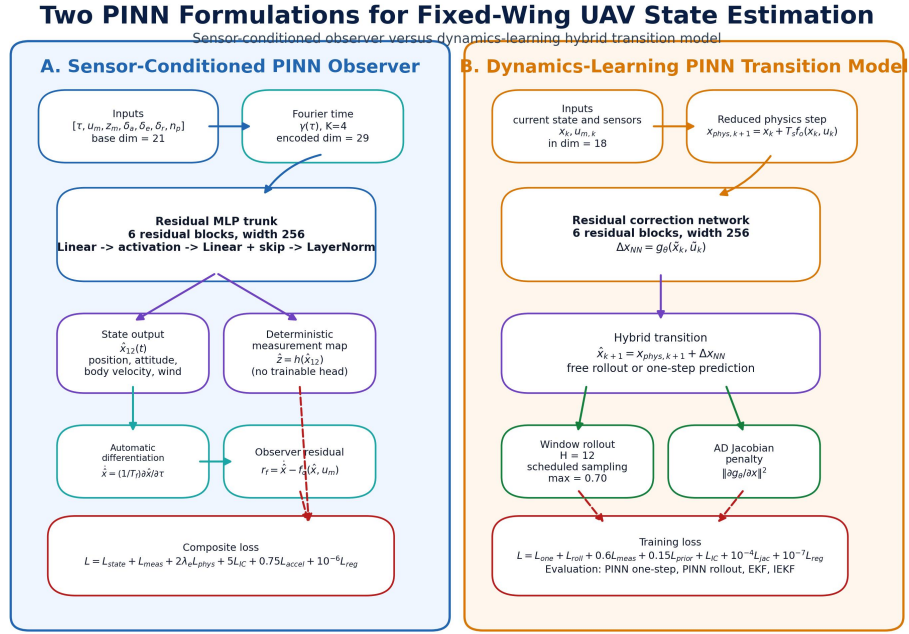
Both implementations use a Huber-style robust mean residual

$$\rho_\delta(r) = \begin{cases} \frac{1}{2}r^2, & |r| \leq \delta, \\ \delta(|r| - \frac{1}{2}\delta), & |r| > \delta, \end{cases} \quad (48)$$

with $\delta = 1$ unless otherwise specified. Let $H(R)$ denote the mean of ρ_δ over a residual matrix.

TABLE I: Principal Configuration Differences Encoded in the Two Scripts

Category	Sensor-conditioned PINN observer	Dynamics-learning PINN transition	Reasoning
Train maneuvers	7: straight-figure-eight	8: straight-figure-eight + spiral	Dynamics model benefits from extra trajectory coverage.
Held-out maneuver	turn	turn	Common generalization test.
State dimension	12	12	Common reduced observer state.
Measurement dimension	10	10	Common measurement map $h(x)$.
Input dimension	21 base, 29 encoded	18	Direct observer uses sensors/controls/time; transition model uses
Fourier features	$K = 4$ time features	none	Direct time-conditioned observer needs time basis; transition model
Hidden width/blocks	256 / 6	256 / 6	Equal capacity for fair architectural comparison.
Adam epochs/phase	350	220	Direct AD residual needs more pointwise refinement.
Activation sweep	40 epochs	35 epochs	Validation-based activation selection.
L-BFGS-B	80 iterations/phase	60 iterations/phase	Deterministic fine tuning after Adam.
Rollout horizon	inactive optional neighbor term	12-sample windows	Transition learning explicitly trains rollout consistency.



Both scripts use the same 12-state reduced observer model, 10-channel measurement map, 20 Hz fixed-wing simulation, robust Huber-style residuals, curriculum learning, Adam pretraining, and L-BFGS-B fine tuning.

Fig. 1: Architecture comparison of the two supplied implementations. The sensor-conditioned PINN maps time, sensors, and controls directly to a 12-state estimate and enforces physics using automatic differentiation in time. The dynamics-learning PINN learns a residual correction to a reduced observer-physics transition and is trained with sequence-window losses.

TABLE II: Noise and Simulation Parameters Shared by the Comparison

Parameter	Value	Applied channel
T_f	120 s	maneuver duration
T_s	0.05 s	20 Hz sampling
Seed	1234	reproducibility
σ_V	0.01	body velocity perturbation
σ_ω	0.001	body-rate perturbation
σ_{wind}	0.0	wind perturbation
σ_{meas}	0.1	10-channel measurement noise
σ_A	0.03	acceleration input noise

B. Sensor-Conditioned PINN Loss

The state residual is computed in normalized coordinates, with yaw residuals wrapped in physical coordinates. With state weights

$$w_x = [10, 10, 10, 3, 3, 10, 6, 6, 2, 2, 2]^T, \quad (49)$$

the state loss is

$$L_{state} = H(r_x \odot w_x). \quad (50)$$

The deterministic measurement prediction $\hat{z} = h(\hat{x})$ gives

$$L_{meas} = H(r_z \odot w_z), \quad w_z = [10, 10, 10, 4, 4, 8, 6, 6, 6, 8]^T. \quad (51)$$

The physics loss is

$$L_{phys} = H(\dot{\hat{x}} - f_0(\hat{x}, u_m)). \quad (52)$$

The acceleration-consistency loss rearranges (13)–(15) to predict measured accelerations:

$$\hat{A}_{mx} = \dot{\hat{u}} + q\hat{w} - r\hat{v} + g \sin \hat{\theta}, \quad (53)$$

$$\hat{A}_{my} = \dot{\hat{v}} - p\hat{w} + r\hat{u} - g \cos \hat{\theta} \sin \hat{\phi}, \quad (54)$$

$$\hat{A}_{mz} = \dot{\hat{w}} + p\hat{v} - q\hat{u} - g \cos \hat{\phi} \cos \hat{\theta}. \quad (55)$$

Then

$$L_{accel} = H((\hat{A}_m - A_m)/\sigma_A). \quad (56)$$

Training and Evaluation Flow from the Two Python Implementations

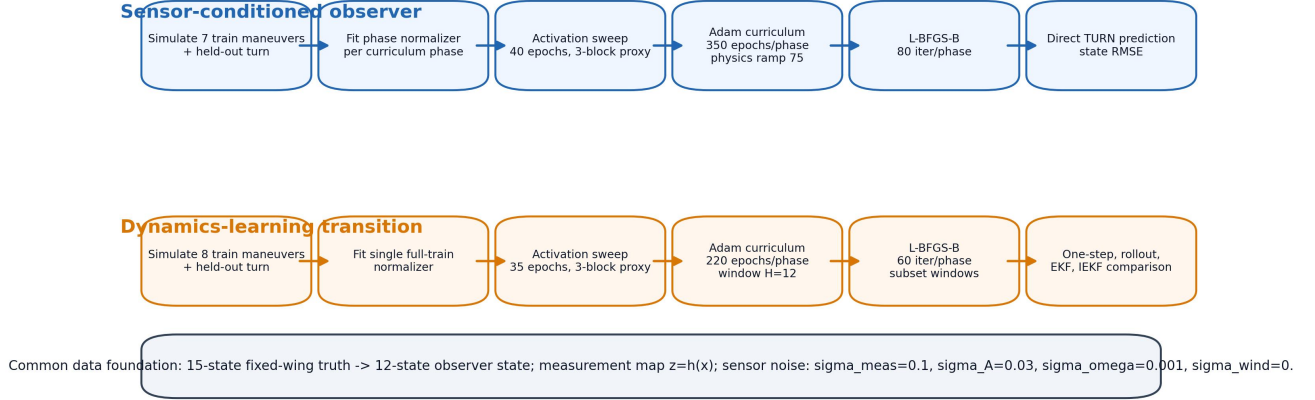


Fig. 2: Training and evaluation flow. Both codes generate synthetic fixed-wing maneuvers, use curriculum training, perform an activation sweep, train with Adam, and fine tune with L-BFGS-B. The dynamics-learning code additionally uses windowed rollout training and compares against EKF/IEKF estimates.

The implemented total loss is

$$L_{SC} = W_{\text{state}}L_{\text{state}} + W_{\text{meas}}L_{\text{meas}} + \lambda_e W_{\text{phys}}L_{\text{phys}} + W_{IC}L_{IC} + W_{\text{roll}}L_{\text{roll}} + W_{\text{accel}}L_{\text{accel}} + W_{\text{reg}}L_{\text{reg}}, \quad (57)$$

where L_{roll} is defined but inactive unless a neighbor batch is supplied. The code uses

$$(W_{\text{state}}, W_{\text{meas}}, W_{\text{phys}}, W_{IC}, W_{\text{roll}}, W_{\text{accel}}, W_{\text{reg}}) = (1, 1, 2, 5, 1, 1, 1), \quad (58)$$

The physics ramp is $\lambda_e = \min(1, (e + 1)/75)$ during Adam training.

C. Dynamics-Learning Loss

The transition model uses one-step prediction, rollout prediction, measurement consistency, correction-size physics prior, initial-condition anchoring, Jacobian smoothness, and weight decay:

$$L_D = W_{\text{one}}L_{\text{one}} + W_{\text{roll}}L_{\text{roll}} + W_{\text{meas}}L_{\text{meas}} + W_{\text{prior}}L_{\text{prior}} + W_{IC}L_{IC} + W_{\text{jac}}L_{\text{jac}} + W_{\text{reg}}L_{\text{reg}}. \quad (59)$$

The implemented weights are

$$(W_{\text{one}}, W_{\text{roll}}, W_{\text{meas}}, W_{\text{prior}}, W_{IC}, W_{\text{jac}}, W_{\text{reg}}) = (1, 1, 0.6, 0.15, 1, 10^{-4}, 10^{-4}). \quad (60)$$

The Jacobian penalty uses automatic differentiation of the neural correction with respect to normalized state input,

$$L_{\text{jac}} \propto \left\| \frac{\partial g_\theta}{\partial \tilde{x}} \right\|_F^2. \quad (61)$$

The use of a 12-step window and scheduled sampling is the key distinction from the pointwise direct observer.

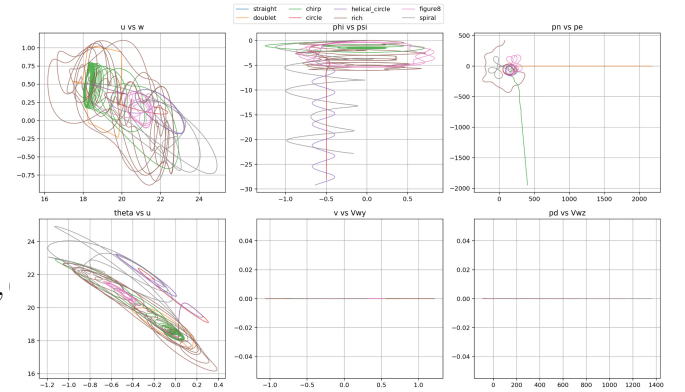


Fig. 3: Training phase portraits from the dynamics-learning code. The training set includes straight, doublet, chirp, circle, helical circle, rich, figure-eight, and spiral maneuvers.

VII. RESULTS

A. Maneuver Coverage

The phase portraits in Figs. 3 and 4 show that both codes generate maneuver-rich data, but the dynamics-learning code includes an additional spiral maneuver. The portraits expose longitudinal, lateral-directional, and coupled state-space regions. Several wind-state portraits remain nearly flat because the simulator uses zero wind and $\sigma_{\text{wind}} = 0$.

B. Held-Out Turn: Sensor-Conditioned Observer

Fig. 5 shows the direct sensor-conditioned state estimates on the held-out turn. Position estimates drift substantially in the inertial channels, while body-velocity and wind channels remain comparatively small in magnitude. The RMSE bar plot in Fig. 6 shows that the position states dominate the aggregate error.

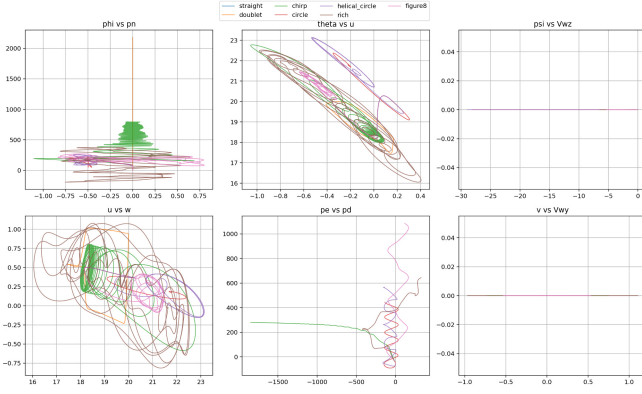


Fig. 4: Training phase portraits from the sensor-conditioned observer code. The training set includes straight, doublet, chirp, circle, helical circle, rich, and figure-eight maneuvers; the turn is held out.

C. Held-Out Turn: Dynamics-Learning PINN and Filters

Fig. 7 compares the dynamics-learning free rollout with EKF and IEKF trajectories. The free rollout follows the position and attitude evolution better than the direct observer in this plotted run, but the wind channels drift significantly. Fig. 8 compares PINN one-step, PINN rollout, EKF, and IEKF RMSE bars. The one-step bars are visually small because one-step prediction uses truth states as inputs, whereas free rollout accumulates model error.

D. Direct PINN-to-PINN RMSE Comparison

Table III reports numerical RMSE values read from the supplied state/RMSE plots. The comparison is intentionally state-wise because the states have different physical units and scales.

VIII. DISCUSSION

A. Why the Dynamics-Learning Model Gives Better Position and Attitude

The dynamics-learning model explicitly propagates a state sequence using the reduced observer model plus a learned correction. This makes it structurally aligned with the temporal evolution of position and attitude. The 12-step rollout loss and scheduled sampling also expose the model to its own prediction distribution during training. Consequently, even when the free rollout accumulates error, it preserves the kinematic structure of the turn better than a pointwise sensor-conditioned map in the supplied result.

B. Why the Sensor-Conditioned Observer Gives Better Wind and Some Velocity Estimates

The sensor-conditioned observer receives the noisy measurement vector z_m at each time sample, including position, attitude, inertial velocity, and airspeed information. Since the deterministic measurement map couples body velocity, inertial velocity, and wind, the measurement-consistency term strongly constrains wind. In contrast, the dynamics-learning transition model uses a zero-wind process model with a learned

correction. During free rollout, small correction errors can accumulate into large wind drift, especially because wind states have zero nominal dynamics.

C. Role of EKF and IEKF Baselines

The dynamics-learning code evaluates EKF and IEKF baselines using the same reduced observer model and measurement map. These baselines are important because they represent recursive estimators with explicit covariance propagation. The neural models in this comparison do not output uncertainty, so they should not be interpreted as replacements for filters without further probabilistic wrapping. A practical system could combine neural residuals with a filter, for example by using a learned correction in the process model while maintaining covariance updates.

D. Novelty of the Comparison

The novelty is not simply the use of a PINN. The comparison isolates two distinct neural-observer philosophies under a common UAV simulation and measurement setup. The first is a sensor-conditioned continuous-time reconstruction PINN. The second is a hybrid transition-learning PINN with rollout loss and classical-filter baselines. Their complementary error patterns show that future methods should combine sensor-conditioned reconstruction with transition consistency rather than choosing only one.

IX. LIMITATIONS

[See below table IV]

X. CONCLUSION

This paper compared two PINN-based UAV state-estimation implementations under a common fixed-wing simulation and held-out turn evaluation. The sensor-conditioned PINN is an observer-like reconstruction map using time, sensor measurements, measured inertial inputs, and controls. The dynamics-learning PINN is a hybrid transition model that learns a correction to the reduced observer physics and can be rolled out sequentially. The supplied plots show a clear tradeoff: the dynamics-learning free rollout gives lower position and attitude RMSE, while the sensor-conditioned observer gives lower body-velocity and wind-component RMSE. These results suggest a next-generation architecture that combines the strengths of both approaches: sensor-conditioned correction inside a recurrent or filtering loop, explicit rollout consistency, uncertainty propagation, and robust measurement updates. Such a hybrid would better connect differentiable physics-informed learning with flight-estimation practice.

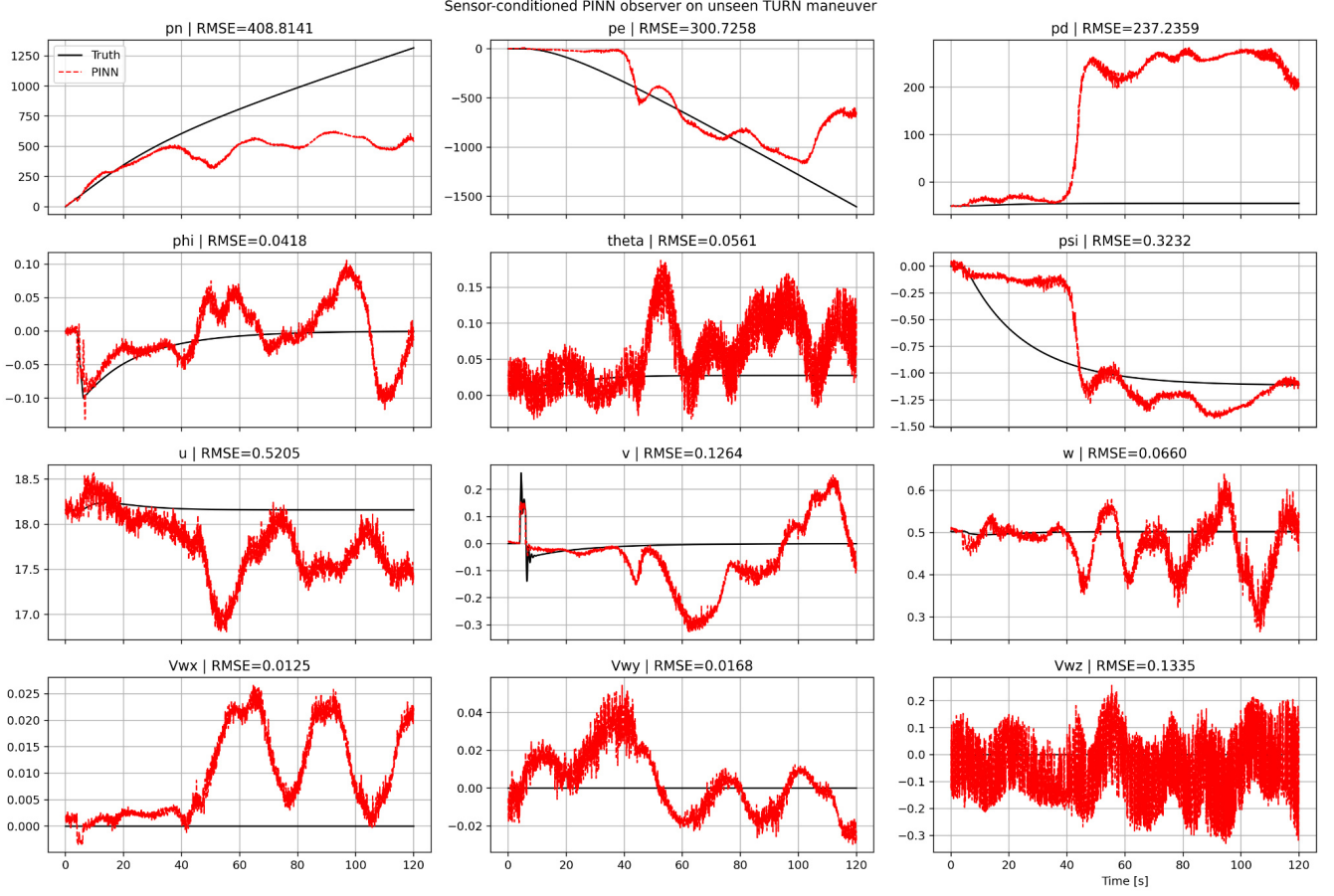


Fig. 5: Sensor-conditioned PINN observer on the unseen turn maneuver. Black curves denote simulator truth and red dashed curves denote direct PINN estimates.

TABLE III: Held-Out Turn RMSE Comparison Between the Two PINN Formulations

State	Unit	Sensor-conditioned PINN	Dynamics-learning PINN rollout	Lower error
p_N	m	408.8141	296.8169	Dynamics rollout
p_E	m	300.7258	225.7450	Dynamics rollout
p_D	m	237.2359	156.2293	Dynamics rollout
ϕ	rad	0.0418	0.0008	Dynamics rollout
θ	rad	0.0561	0.0012	Dynamics rollout
ψ	rad, wrapped	0.3232	0.0013	Dynamics rollout
u	m/s	0.5205	0.6032	Sensor-conditioned
v	m/s	0.1264	0.3102	Sensor-conditioned
w	m/s	0.0660	0.2185	Sensor-conditioned
V_{wx}	m/s	0.0125	4.6457	Sensor-conditioned
V_{wy}	m/s	0.0168	6.3463	Sensor-conditioned
V_{wz}	m/s	0.1335	5.0509	Sensor-conditioned

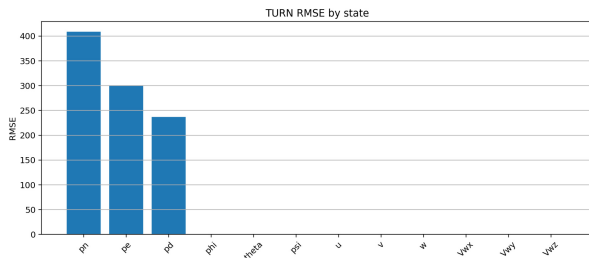


Fig. 6: State-wise RMSE for the sensor-conditioned PINN observer on the held-out turn maneuver.

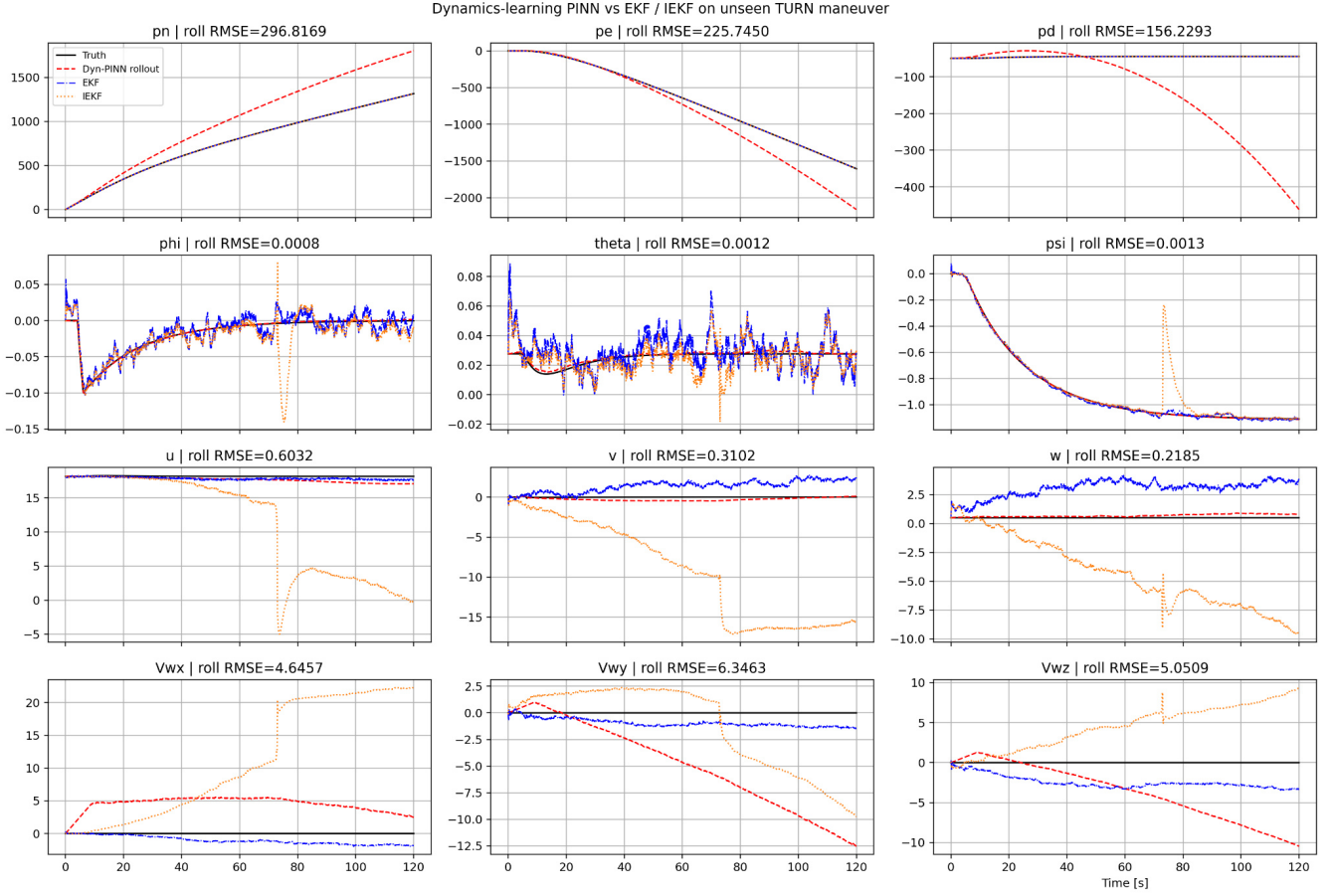


Fig. 7: Dynamics-learning PINN rollout compared with EKF and IEKF estimates on the unseen turn maneuver. Subplot titles report the free-rollout PINN RMSE.

TABLE IV: Limitations and Suggested Mitigations

Limitation	Effect on interpretation	Suggested mitigation
Synthetic data only	Results depend on a deterministic simulator and idealized noise assumptions.	Validate on flight logs; add actuator, bias, delay, and wind randomization.
No learned uncertainty	Neural outputs are point estimates, unlike EKF/IEKF covariance outputs.	Use ensembles, Bayesian layers, or a filter wrapper around learned residuals.
Euler-angle representation	Singularity avoided only by clamping $\cos \theta$.	Use quaternions or Lie-group attitude representation.
Pointwise direct observer	Sensor-conditioned model can drift in position without explicit sequence constraints.	Activate rollout/neighbor loss or train with temporal windows.
Free-rollout accumulation	Dynamics-learning model can accumulate correction errors, especially in wind states.	Add wind priors, measurement updates during rollout, and multi-horizon validation.
Computational cost	AD over 12 outputs and EKF numerical Jacobians are expensive.	Use vector-Jacobian products, mixed precision, sparse collocation, and analytic Jacobians.
Metric scale mismatch	Position, attitude, velocity, and wind units cannot be collapsed into one unweighted RMSE.	Report state-wise and group-wise metrics with unit-aware weighting.

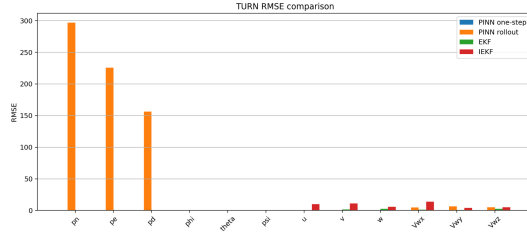


Fig. 8: TURN RMSE comparison from the dynamics-learning code. The plot includes PINN one-step, PINN rollout, EKF, and IEKF bars.

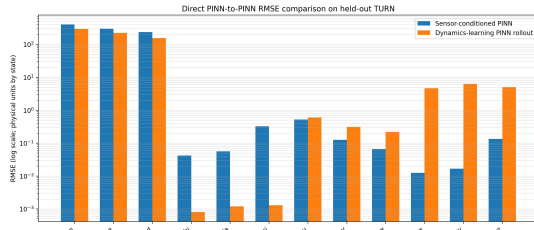


Fig. 9: Direct log-scale RMSE comparison between the sensor-conditioned PINN and the dynamics-learning free rollout. A log scale is used because position, angle, velocity, and wind errors occupy different numerical ranges.

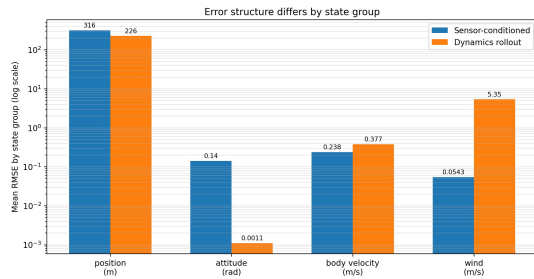


Fig. 10: Mean RMSE by state group. The dynamics-learning rollout is stronger for position and attitude in this run, whereas the sensor-conditioned observer is stronger for body-velocity and wind components.

REFERENCES

- [1] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational Physics*, vol. 378, pp. 686–707, 2019.
- [2] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, and L. Yang, "Physics-informed machine learning," *Nature Reviews Physics*, vol. 3, no. 6, pp. 422–440, 2021.
- [3] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.
- [4] D. Simon, *Optimal State Estimation: Kalman, H Infinity, and Nonlinear Approaches*. Wiley, 2006.
- [5] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud, "Neural ordinary differential equations," in *Advances in Neural Information Processing Systems*, vol. 31, 2018.
- [6] M. Tancik *et al.*, "Fourier features let networks learn high frequency functions in low dimensional domains," in *Advances in Neural Information Processing Systems*, vol. 33, pp. 7537–7547, 2020.
- [7] V. Sitzmann, J. N. P. Martel, A. W. Bergman, D. B. Lindell, and G. Wetzstein, "Implicit neural representations with periodic activation functions," in *Advances in Neural Information Processing Systems*, vol. 33, pp. 7462–7473, 2020.
- [8] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition*, pp. 770–778, 2016.
- [9] B. L. Stevens, F. L. Lewis, and E. N. Johnson, *Aircraft Control and Simulation: Dynamics, Controls Design, and Autonomous Systems*, 3rd ed. Wiley, 2015.
- [10] B. Etkin and L. D. Reid, *Dynamics of Flight: Stability and Control*, 3rd ed. Wiley, 1996.
- [11] R. W. Beard and T. W. McLain, *Small Unmanned Aircraft: Theory and Practice*. Princeton University Press, 2012.
- [12] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations*, 2015.
- [13] R. H. Byrd, P. Lu, J. Nocedal, and C. Zhu, "A limited memory algorithm for bound constrained optimization," *SIAM Journal on Scientific Computing*, vol. 16, no. 5, pp. 1190–1208, 1995.
- [14] Provided Python implementation, "Sensor-conditioned PINN observer for UAV state estimation," uploaded source code, 2026.
- [15] Provided Python implementation, "Dynamics-learning PINN for UAV maneuver generalization," uploaded source code, 2026.